Garry Hannah 02/22/26

CS 250 Mod 7

Sprint Review and Retrospective

In the SNHU Travel project, I took on three different roles on the team. The first role was the Product Owner. I managed the requirements provided by SNHU Travel and decided which features to build first. I put together user stories. Finding the best health vacation packages by location was important when helping clients find the right trips. This helped the team understand what to build and why it mattered to potential customers.

The second role was the Scrum Master, I helped the team work together and fix problems that got in their way. When we had trouble setting up the order processing system, I spoke with the vendor who provided it and worked with other team members to resolve the issue quickly. This job taught me how important it is to help your team by clearing obstacles out of their path.

The third role was as a Development Team member, where I wrote code and tested features. I built a search tool that lets people find vacation packages. The team worked together to review each other's code and make sure everything worked correctly. All three jobs worked together to help our team succeed.

The Scrum-Agile method helped us finish tasks by breaking the big project into smaller pieces. Instead of trying to build everything at once, we worked on the most important features first during two-week work periods called sprints. As Cobb (2015) explains, "User stories are not intended to be a complete specification… They are meant to be 'just barely good enough' to start a conversation." This approach allowed our team to start working quickly without extensive documentation.

For example. As a customer, I want to see details about vacation packages, such as prices and reviews, so I can decide which one to buy. We built a simple version first, then added more features as we received feedback.

Every day, we had 15-minute meetings where, as the Scrum Training Series describes, "each person explains: what they've accomplished… what they plan to accomplish… and what is in their way." This helped us stay on track and finish our tasks. These daily standups helped us "synchronize their work and pull together to meet the Sprint goal" (Scrum Training Series). We also had clear rules for when something was really "done." As Cobb (2015) explains, "Acceptance criteria define the boundaries of a user story and are

used to confirm when a story is completed and working as intended.” The code had to work, be tested, and reviewed by others.

At the end of each two-week sprint, we showed our work to the client and got their feedback. When we showed them the search feature, they asked us to add a price filter, so we added it to our list for the next sprint. This helped make sure we were building what they actually needed.

The Scrum-Agile method helped when things changed unexpectedly. Halfway through the project, the third-party payment processor we planned to use closed. We needed to change the payment system we were using. This was a big change that would have caused problems if we were following the waterfall. As Ambler (n.d.) notes, "Agile software development teams embrace change, accepting the idea that requirements will evolve throughout an initiative."

But because we were using Agile, we could make quick adjustments. We created new tasks to integrate the new payment system. Some less important features got pushed back, and the payment integration work got added to our next sprint.

We ran into another challenge when two team members left for different jobs. Instead of letting this stop everything, we had an emergency meeting where the remaining team members split up the work. We didn't finish everything we planned, but we still delivered the working software.

The Agile approach focuses on adapting to change rather than sticking to a plan. When things change, we can also change. Because our work periods were only two weeks long, we were never far from being able to adjust our plans.

Good communication was important for our project’s success. As Cobb (2015) states, “The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.” In our daily standup meetings, everyone answered three questions: What did I do yesterday? What will I do today? What’s blocking me? This kept everyone informed and helped us solve problems quickly.

I would use images and diagrams to explain tasks. When we talked about the booking process, I found that diagrams showing how the website, server, and payment system worked together were more effective than written words.

I always made sure I sent emails to update everyone after our review meetings. I included screenshots of the new features and documented any decisions we made. This kept

everyone in the loop, even if they couldn't attend the meeting. Cobb (2015) emphasizes that Agile communication works best when information is "highly visible and easily accessible to anyone who needs it" (p. 164).

When we meet to review what went well and what could improve, I encourage everyone to be honest. We make sure to celebrate our wins, big or small.

The main tool we used was Jira, which let us track and organize our tasks into sprints. We had a board with columns like "To Do," "In Progress," "Testing," and "Done" so everyone could see what was happening. Cobb (2015) explains that "The use of visual controls is a very important part of Agile project management for providing transparency and for identifying bottlenecks in the process" (p. 165).

Jira helped us estimate how much work we could get done each day. This helped us plan better. We also had charts that showed if we were falling behind, so we could fix problems early.

During our daily meetings, Jira helped us see which tasks were being completed and which needed more time. These meetings lasted only 15 minutes.

In our review meetings, we showed shared working software, and the Stakeholders could test and try the features, which helped us get better feedback. As Ambler (n.d.) notes, "Successful teams will deploy a working copy of their system at the end of each iteration into a demo environment... This provides another opportunity for feedback, often generating new or improved requirements."

We used Miro to collect feedback on virtual sticky notes. Some people felt more comfortable sharing honestly when it was anonymous. We tracked all our improvement ideas in Jira to ensure we implemented them.

Using these tools in our meetings helped us stay on track, check our progress, and keep improving.

The good thing about using Scrum-Agile in the SNHU Travel project is that we could have working software early in the project, which made stakeholders happy. When requirements changed, we could adapt without ruining the whole project. As Ambler (n.d.) notes, "Agile software development teams embrace change, accepting the idea that requirements will evolve throughout an initiative." Regular review meetings kept everyone aligned on what we were building.

Working as a team improved communication and problem-solving. Two-week sprints helped with productivity.

The downside is that not planning everything upfront can sometimes lead to extra work. If we had planned everything up front, we might have realized that the payment processor we planned to use was unreliable and chosen something else.

All the meetings, like daily standups, planning, and reviews, took up time that could have been spent on coding. For teams in different locations or time zones, scheduling these meetings was challenging.

I still think Scrum-Agile was the best approach for the SNHU Travel project. The project had changing requirements, needed fast delivery, and operated in a competitive market. The travel industry changes quickly, and Agile lets us respond to those changes better.

SNHU Travel wanted to make fast progress. The waterfall methods would have meant waiting months to see the progress. With Agile, we delivered a working product after just a few sprints.

The SNHU Travel project was always going to change as trends changed, so we couldn't know all the requirements upfront. And stakeholders only figured out what they really needed after seeing and using working software. As Ambler (n.d.) explains, "Successful teams will deploy a working copy of their system at the end of each iteration into a demo environment… This provides another opportunity for feedback, often generating new or improved requirements." Agile's approach of transparency and adaptation was perfect for this kind of project.

I think that if we had requirements we had to stick to and everything had to be documented up front, or the clients couldn't meet up for updates, then waterfall might have been a better option. But for the SNHU Travel project, Scrum-Agile was the right choice for delivering this software that met changing needs while keeping the team happy.

REFERENCES

Project Management Docs. (n.d.). *Agile team charter template*. https://projectmanagementdocs.com/template/agile-templates/agile-team-charter/charter/charter/

Scrum Training Series. (2010). *The Daily Scrum meeting* [Video]. https://scrumtrainingseries.com/DailyScrumMeeting/index.htmlhttps://scrumtrainingseries.com/DailyScrumMeeting/index.html

Cobb, C. G. (2015). *The project manager's guide to mastering Agile: Principles and practices for an adaptive approach* (2nd ed.). Wiley.

Leather, E. (2017, April 7). *Amazonian Agility: The A to Z of Agile fuelling the future of Amazon*. Medium. https://medium.com/frontira/amazonian-agility-e3720ff004f7https://medium.com/frontira/amazonian-agility-e3720ff004f7

Ambler, S. W. (n.d.). *Agile requirements change management*. Agile Modeling. https://agilemodeling.com/essays/changeManagement.htm